

Computer Organization and Design

Chapter 4

Reference: Computer Systems Architecture

M. Morris Mano

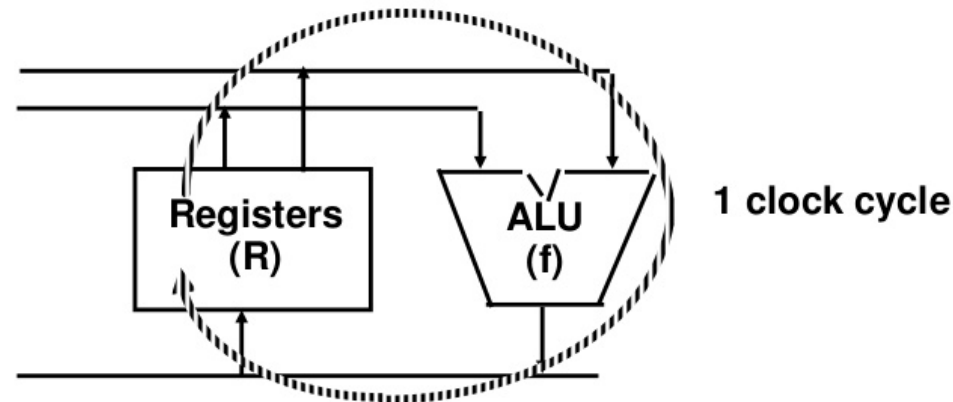
REGISTER TRANSFER AND MICROOPERATIONS

- **Register Transfer Language**
- **Register Transfer**
- **Bus and Memory Transfers**
- **Arithmetic Microoperations**
- **Logic Microoperations**
- **Shift Microoperations**
- **Arithmetic Logic Shift Unit**

MICROOPERATIONS (1)

- The operations on the data in registers are called microoperations.
- The functions built into registers are examples of microoperations
 - Shift
 - Load
 - Clear
 - Increment
 - ...

An elementary operation performed (during one clock pulse), on the information stored in one or more registers



$$R \leftarrow f(R, R)$$

f: shift, load, clear, increment, add, subtract, complement, and, or, xor, ...

ORGANIZATION OF A DIGITAL SYSTEM

- **Definition of the (internal) organization of a computer**
 - **Set of registers and their functions**
 - **Microoperations set**

Set of allowable microoperations provided by the organization of the computer
 - **Control signals that initiate the sequence of microoperations (to perform the functions)**

REGISTER TRANSFER LANGUAGE

- Rather than specifying a digital system in words, a specific notation is used, *register transfer language*
- For any function of the computer, the register transfer language can be used to describe the (sequence of) microoperations
- Register transfer language
 - A symbolic language
 - A convenient tool for describing the internal organization of digital computers
 - Can also be used to facilitate the design process of digital systems.
- Registers are designated by capital letters, sometimes followed by numbers (e.g., A, R13, IR)
- Often the names indicate function:
 - MAR - memory address register
 - PC - program counter
 - IR - instruction register
- Registers and their contents can be viewed and represented in *various ways*
 - A register can be viewed as a single entity:

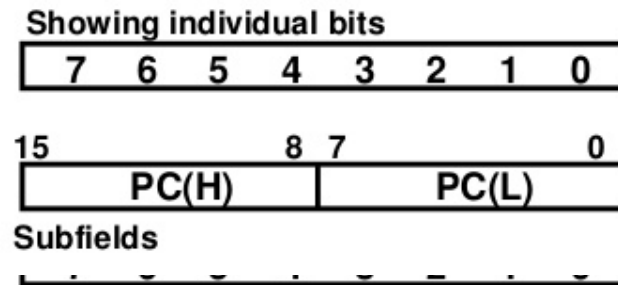
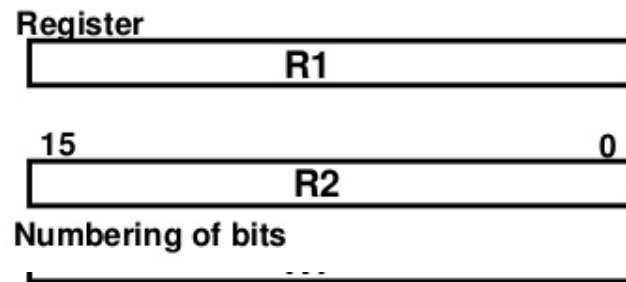


MAR

- Registers may also be represented showing the bits of data they contain

DESIGNATION OF REGISTERS

- Designation of a register
 - a register
 - portion of a register
 - a bit of a register
- Common ways of drawing the block diagram of a register



- Copying the contents of one register to another is a register⁰ transfer
- A register transfer is indicated as
 - In this case the contents of register R2 are copied (loaded) into register R1
 - A simultaneous transfer of all bits from the source R1 to the destination register R2, during one clock pulse
 - Note that this is a non-destructive; i.e. the contents of R1 are not altered by copying (loading) them to R2

R2 ← R1

CONTROL FUNCTIONS

- Often actions need to only occur if a certain condition is true
- This is similar to an “if” statement in a programming language
- In digital systems, this is often done via a *control signal*, called a *control function*
 - If the signal is 1, the action takes place
- This is represented as:

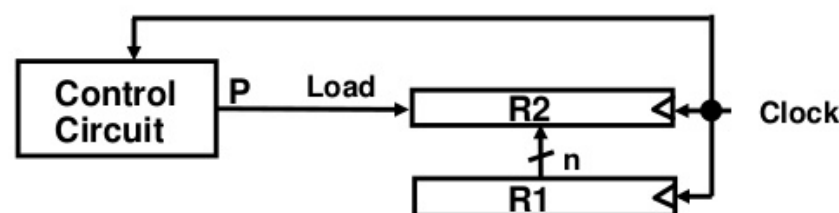
P: $R2 \leftarrow R1$ P: $R3 \leftarrow R5, MAR \leftarrow IR$

Which means “if $P = 1$, then load the contents of register $R1$ into register $R2$ ”, i.e., if $(P = 1)$ then $(R2 \leftarrow R1)$

Implementation of controlled transfer

P: $R2 \leftarrow R1$

Block diagram



BASIC SYMBOLS FOR REGISTER TRANSFERS

Symbols	Description	Examples
Capital letters & numerals	Denotes a register	MAR, R2
Parentheses ()	Denotes a part of a register	R2(0-7), R2(L)
Arrow ←	Denotes transfer of information	R2 ← R1
Colon :	Denotes termination of control function	P:
Comma ,	Separates two micro-operations	A ← B, B ← A

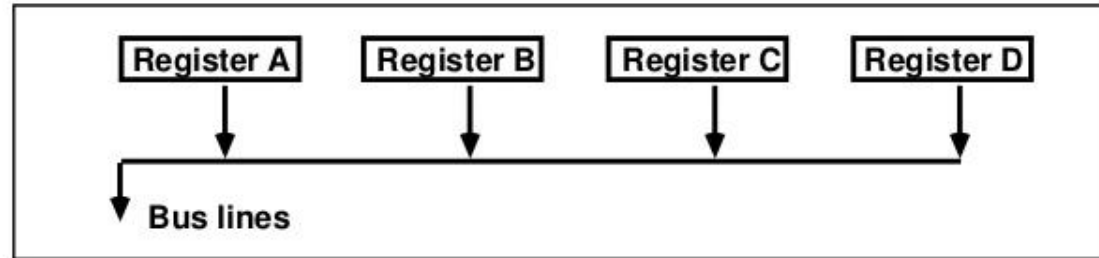
CONNECTING REGISTRS

- In a digital system with many registers, it is impractical to have data and control lines to directly allow each register to be loaded with the contents of every possible other registers
- Instead, take a different approach
- Have one centralized set of circuits for data transfer – the **bus**
- Have control circuits to select which register is the source, and which is the destination

BUS AND BUS TRANSFER

Bus is a path(of a group of wires) over which information is transferred, from any of several sources to any of several destinations.

From a register to bus: $BUS \leftarrow R$

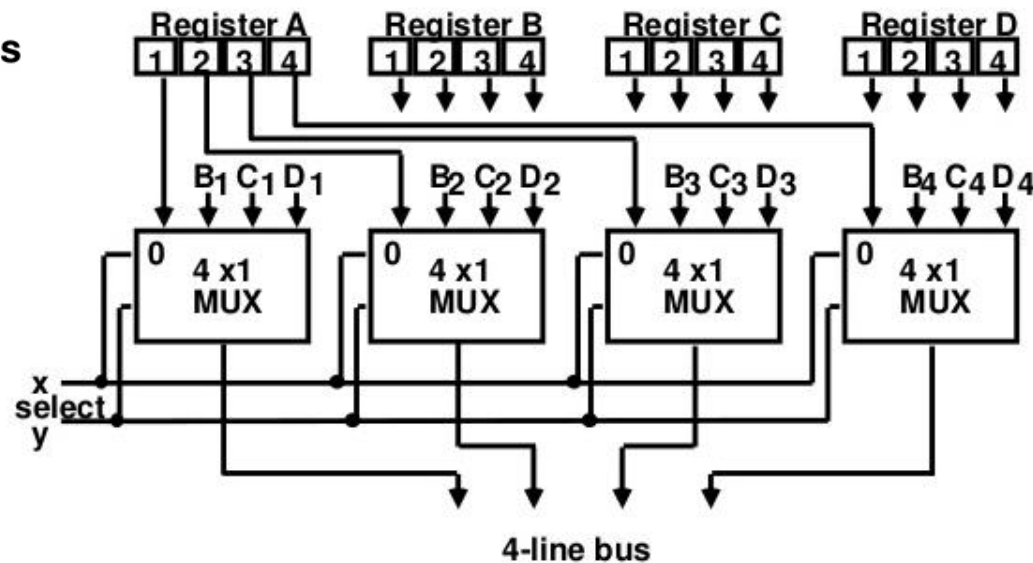


Depending on whether the bus is to be mentioned explicitly or not, register transfer can be indicated as either

$R2 \leftarrow R1$

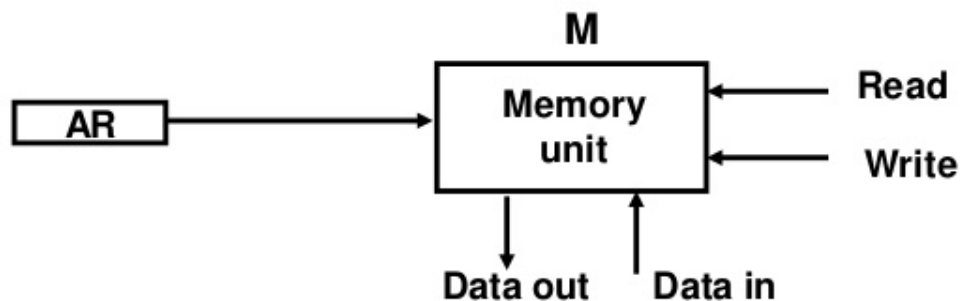
or

$BUS \leftarrow R1, R2 \leftarrow BUS$



MEMORY TRANSFER

- Collectively, the memory is viewed at the register level as a device, M.
- Since it contains multiple locations, we must specify which address in memory we will be using
- This is done by indexing memory references
- Memory is usually accessed in computer systems by putting the desired address in a special register, the *Memory Address Register (MAR, or AR)*
- When memory is accessed, the contents of the MAR get sent to the memory unit's address lines



MEMORY READ

- To read a value from a location in memory and load it into a register, the register transfer language notation looks like this:

$R1 \leftarrow M[MAR]$

- This causes the following to occur
 - The contents of the MAR get sent to the memory address lines
 - A Read (= 1) gets sent to the memory unit
 - The contents of the specified address are put on the memory's output data lines
 - These get sent over the bus to be loaded into register R1

MEMORY WRITE

- To write a value from a register to a location in memory looks like this in register transfer language:

$M[MAR] \leftarrow R1$

- This causes the following to occur
 - The contents of the MAR get sent to the memory address lines
 - A Write (= 1) gets sent to the memory unit
 - The values in register R1 get sent over the bus to the data input lines of the memory
 - The values get loaded into the specified address in the memory

MICROOPERATIONS

• Computer system microoperations are of four types:

- Register transfer microoperations
- Arithmetic microoperations
- Logic microoperations
- Shift microoperations

Summary of Typical Arithmetic Micro-Operations

$R3 \leftarrow R1 + R2$	Contents of R1 plus R2 transferred to R3
$R3 \leftarrow R1 - R2$	Contents of R1 minus R2 transferred to R3
$R2 \leftarrow R2'$	Complement the contents of R2
$R2 \leftarrow R2' + 1$	2's complement the contents of R2 (negate)
$R3 \leftarrow R1 + R2' + 1$	subtraction
$R1 \leftarrow R1 + 1$	Increment
$R1 \leftarrow R1 - 1$	Decrement

16 logic micro-operations

Boolean Function	Micro-Operations	Name
$F0 = 0$	$F \leftarrow 0$	Clear
$F1 = xy$	$F \leftarrow A \wedge B$	AND
$F2 = xy'$	$F \leftarrow A \wedge B'$	Transfer A
$F3 = x$	$F \leftarrow A$	
$F4 = x'y$	$F \leftarrow A' \wedge B$	Transfer B
$F5 = y$	$F \leftarrow B$	
$F6 = x \oplus y$	$F \leftarrow A \oplus B$	Exclusive-OR
$F7 = x + y$	$F \leftarrow A \vee B$	OR
$F8 = (x + y)'$	$F \leftarrow (A \vee B)'$	NOR
$F9 = (x \oplus y)'$	$F \leftarrow (A \oplus B)'$	Exclusive-NOR
$F10 = y'$	$F \leftarrow B'$	Complement B
$F11 = x + y'$	$F \leftarrow A \vee B$	Complement A
$F12 = x'$	$F \leftarrow A'$	
$F13 = x' + y$	$F \leftarrow A' \vee B$	NAND
$F14 = (xy)'$	$F \leftarrow (A \wedge B)'$	
$F15 = 1$	$F \leftarrow \text{all 1's}$	Set to all 1's

APPLICATIONS OF LOGIC MICROOPERATIONS

- Logic microoperations can be used to manipulate individual bits or a portions of a word in a register
- Consider the data in a register A. In another register, B, is bit data that will be used to modify the contents of A

– Selective-set	$A \leftarrow A + B$
– Selective-complement	$A \leftarrow A \oplus B$
– Selective-clear	$A \leftarrow A \cdot B'$
– Mask (Delete)	$A \leftarrow A \cdot B$
– Clear	$A \leftarrow A \oplus B$
– Insert	$A \leftarrow (A \cdot B) + C$
– Compare	$A \leftarrow A \oplus B$

Chapter 5

INTRODUCTION

- Every different processor type has its own design (different registers, buses, microoperations, machine instructions, etc)
- Modern processor is a very complex device
- It contains
 - Many registers
 - Multiple arithmetic units, for both integer and floating point calculations
 - The ability to pipeline several consecutive instructions to speed execution
 - Etc.
- However, to understand how processors work, we will start with a simplified processor model
- This is similar to what real processors were like ~25 years ago
- M. Morris Mano introduces a simple processor model he calls the *Basic Computer*
- We will use this to introduce processor organization and the relationship of the RTL model to the higher level computer processor

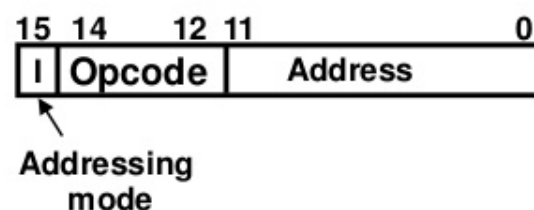
INSTRUCTIONS

- **Program**
 - A sequence of (machine) instructions
- **(Machine) Instruction**
 - A group of bits that tell the computer to *perform a specific operation* (a sequence of micro-operation)
- The instructions of a program, along with any needed data are stored in memory
- The CPU reads the next instruction from memory
- It is placed in an *Instruction Register (IR)*
- Control circuitry in control unit then translates the instruction into the sequence of microoperations necessary to implement it

INSTRUCTION FORMAT

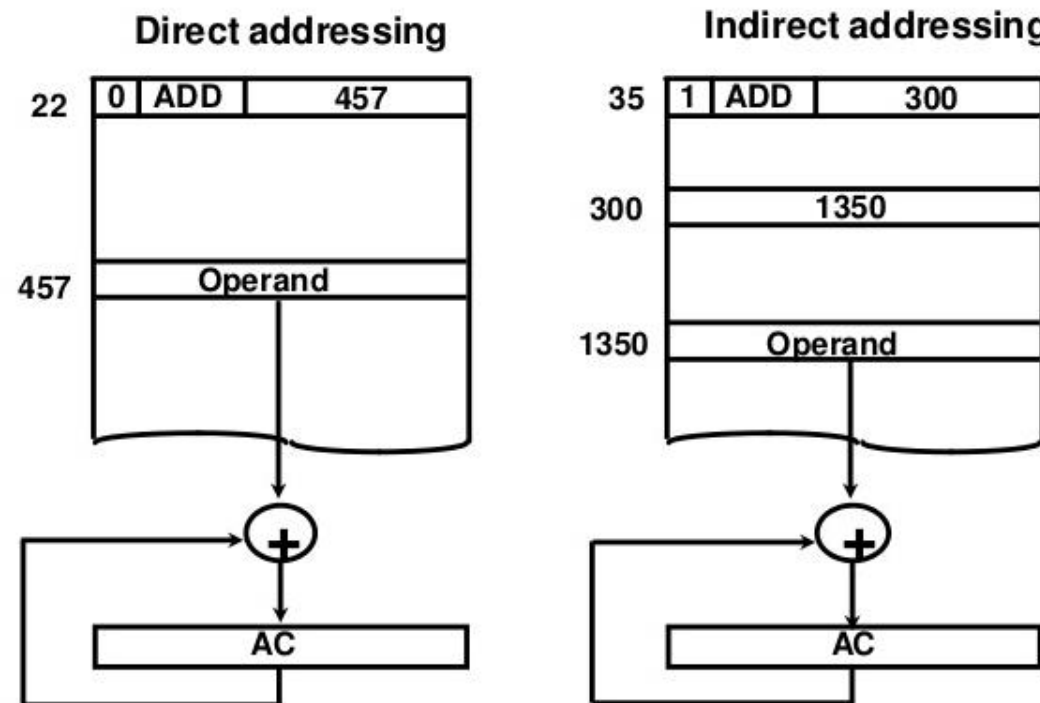
- A computer instruction is often divided into two parts
 - An *opcode* (Operation Code) that specifies the operation for that instruction
 - An *address* that specifies the registers and/or locations in memory to use for that operation
- In the Basic Computer, since the memory contains 4096 ($= 2^{12}$) words, we need 12 bit to specify which memory address this instruction will use
- In the Basic Computer, bit 15 of the instruction specifies the *addressing mode* (0: direct addressing, 1: indirect addressing)
- Since the memory words, and hence the instructions, are 16 bits long, that leaves 3 bits for the instruction's opcode

Instruction Format



ADDRESSING MODES

- The address field of an instruction can represent either
 - Direct address: the address in memory of the data to use (the address of the operand), or
 - Indirect address: the address in memory of the address in memory of the data to use



- **Effective Address (EA)**
 - The address, that can be directly used without modification to access an operand for a computation-type instruction, or as the target address for a branch-type instruction

PROCESSOR REGISTERS

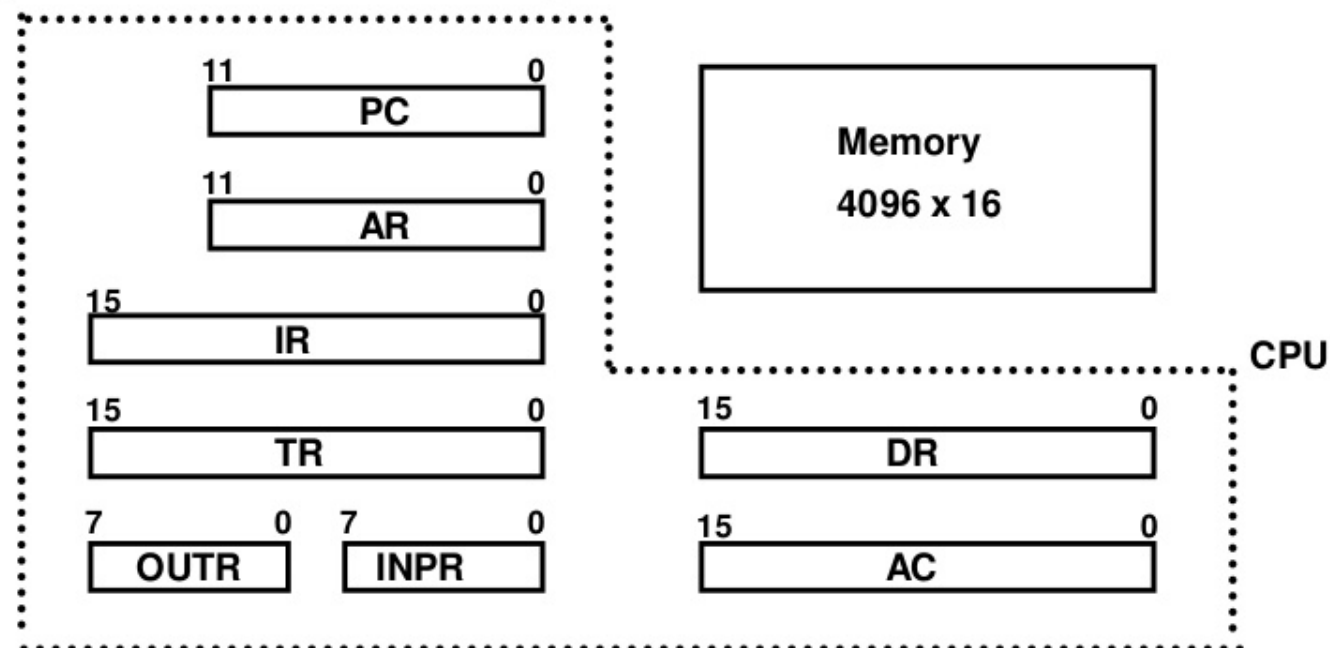
- A processor has many registers to hold instructions, addresses, data, etc
- The processor has a register, the *Program Counter* (**PC**) that holds the memory address of the next instruction to get
 - Since the memory in the Basic Computer only has 4096 locations, the PC only needs 12 bits
- In a direct or indirect addressing, the processor needs to keep track of what locations in memory it is addressing: The *Address Register* (**AR**) is used for this
 - The AR is a 12 bit register in the Basic Computer
- When an operand is found, using either direct or indirect addressing, it is placed in the *Data Register* (**DR**). The processor then uses this value as data for its operation
- The Basic Computer has a single *general purpose register* – the *Accumulator* (**AC**)

PROCESSOR REGISTERS

- The significance of a general purpose register is that it can be referred to in instructions
 - e.g. load AC with the contents of a specific memory location; store the contents of AC into a specified memory location
- Often a processor will need a scratch register to store intermediate results or other temporary data; in the Basic Computer this is the *Temporary Register* (TR)
- The Basic Computer uses a very simple model of input/output (I/O) operations
 - Input devices are considered to send 8 bits of character data to the processor
 - The processor can send 8 bits of character data to output devices
- The *Input Register* (INPR) holds an 8 bit character gotten from an input device
- The *Output Register* (OUTR) holds an 8 bit character to be send to an output device

BASIC COMPUTER REGISTERS

Registers in the Basic Computer



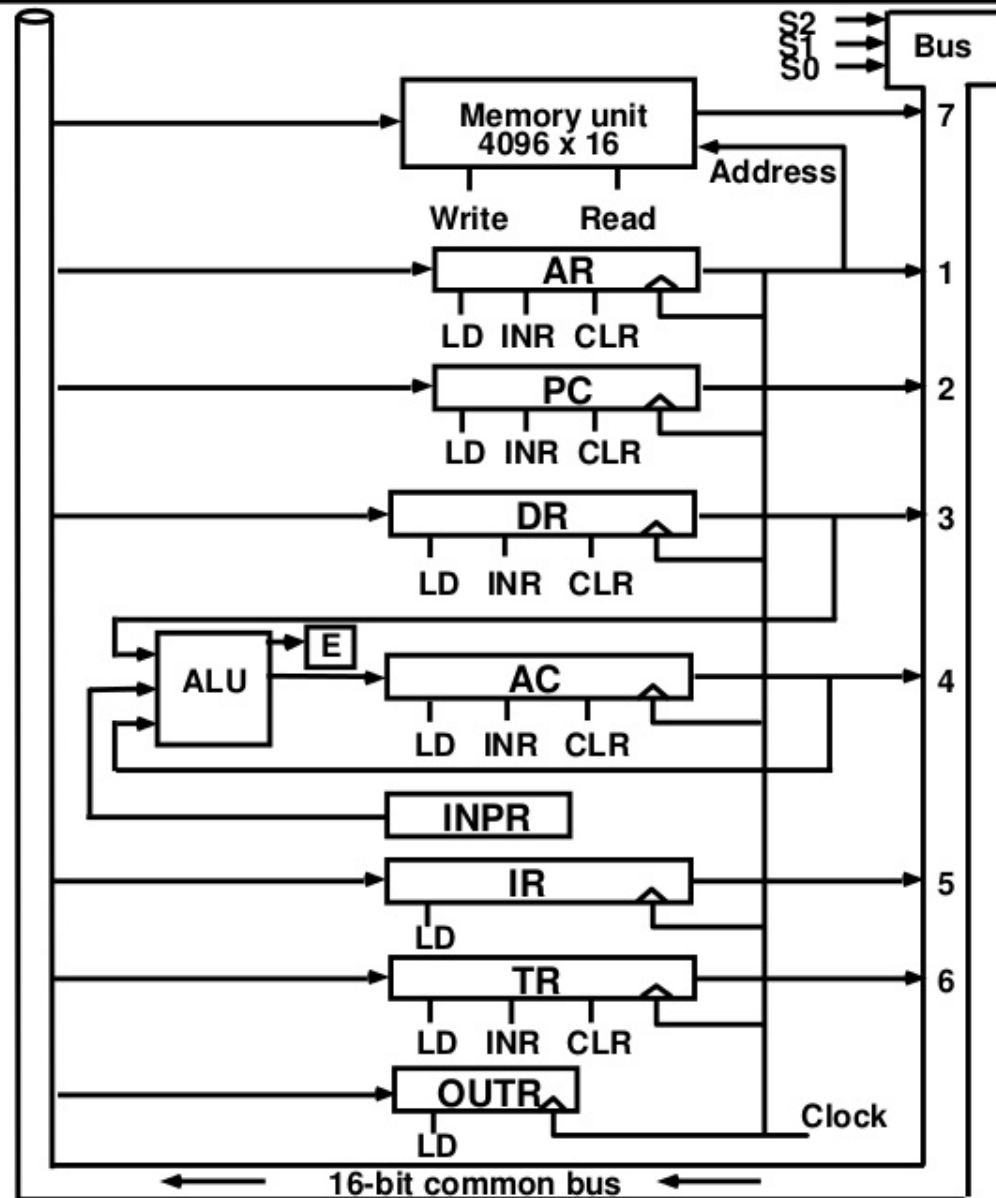
List of BC Registers

DR	16	Data Register	Holds memory operand
AR	12	Address Register	Holds address for memory
AC	16	Accumulator	Processor register
IR	16	Instruction Register	Holds instruction code
PC	12	Program Counter	Holds address of instruction
TR	16	Temporary Register	Holds temporary data
INPR	8	Input Register	Holds input character
OUTR	8	Output Register	Holds output character

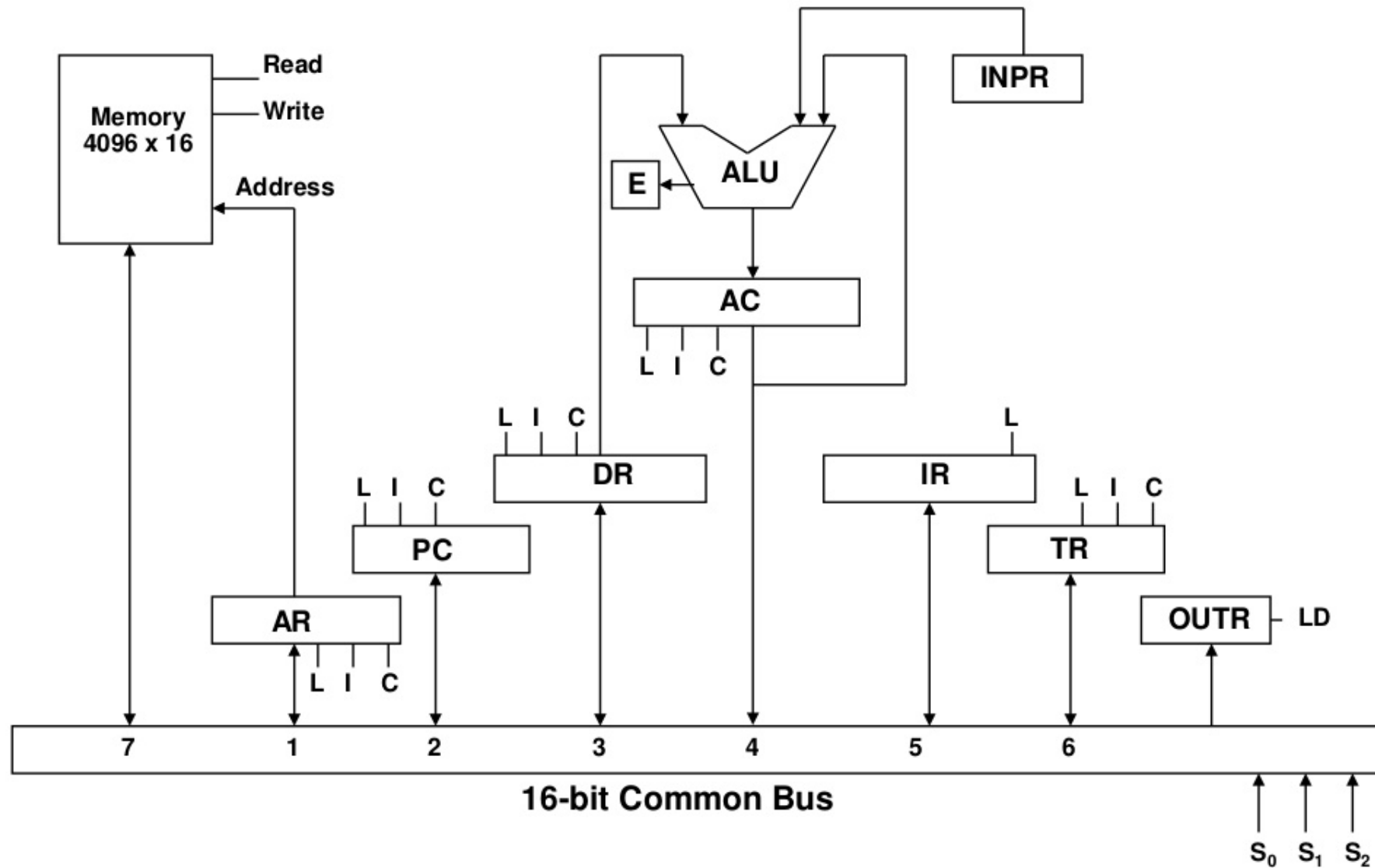
COMMON BUS SYSTEM

- The registers in the Basic Computer are connected using a bus
- This gives a savings in circuitry over complete connections between registers

COMMON BUS SYSTEM



COMMON BUS SYSTEM



COMMON BUS SYSTEM

- Three control lines, S_2 , S_1 , and S_0 control which register the bus selects as its input

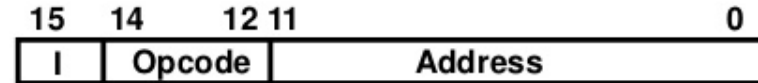
S_2	S_1	S_0	Register
0	0	0	x
0	0	1	AR
0	1	0	PC
0	1	1	DR
1	0	0	AC
1	0	1	IR
1	1	0	TR
1	1	1	Memory

- Either one of the registers will have its load signal activated, or the memory will have its read signal activated
 - Will determine where the data from the bus gets loaded
- The 12-bit registers, AR and PC, have 0's loaded onto the bus in the high order 4 bit positions
- When the 8-bit register OTR is loaded from the bus, the data comes from the low order 8 bits on the bus

BASIC COMPUTER INSTRUCTIONS

- Basic Computer Instruction Format

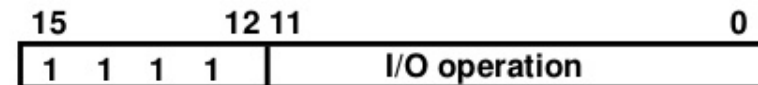
Memory-Reference Instructions (OP-code = 000 ~ 110)



Register-Reference Instructions (OP-code = 111, I = 0)



Input-Output Instructions (OP-code = 111, I = 1)



BASIC COMPUTER INSTRUCTIONS

Symbol	Hex Code		Description
	I = 0	I = 1	
AND	0xxx	8xxx	AND memory word to AC
ADD	1xxx	9xxx	Add memory word to AC
LDA	2xxx	Axxx	Load AC from memory
STA	3xxx	Bxxx	Store content of AC into memory
BUN	4xxx	Cxxx	Branch unconditionally
BSA	5xxx	Dxxx	Branch and save return address
ISZ	6xxx	Exxx	Increment and skip if zero
CLA	7800		Clear AC
CLE	7400		Clear E
CMA	7200		Complement AC
CME	7100		Complement E
CIR	7080		Circulate right AC and E
CIL	7040		Circulate left AC and E
INC	7020		Increment AC
SPA	7010		Skip next instr. if AC is positive
SNA	7008		Skip next instr. if AC is negative
SZA	7004		Skip next instr. if AC is zero
SZE	7002		Skip next instr. if E is zero
HLT	7001		Halt computer
INP	F800		Input character to AC
OUT	F400		Output character from AC
SKI	F200		Skip on input flag
SKO	F100		Skip on output flag
ION	F080		Interrupt on
IOF	F040		Interrupt off

INSTRUCTION SET COMPLETENESS

A computer should have a set of instructions so that the user can construct machine language programs to evaluate any function that is known to be computable.

- **Instruction Types**

- Functional Instructions**

- Arithmetic, logic, and shift instructions
 - ADD, CMA, INC, CIR, CIL, AND, CLA

- Transfer Instructions**

- Data transfers between the main memory and the processor registers
 - LDA, STA

- Control Instructions**

- Program sequencing and control
 - BUN, BSA, ISZ

- Input/Output Instructions**

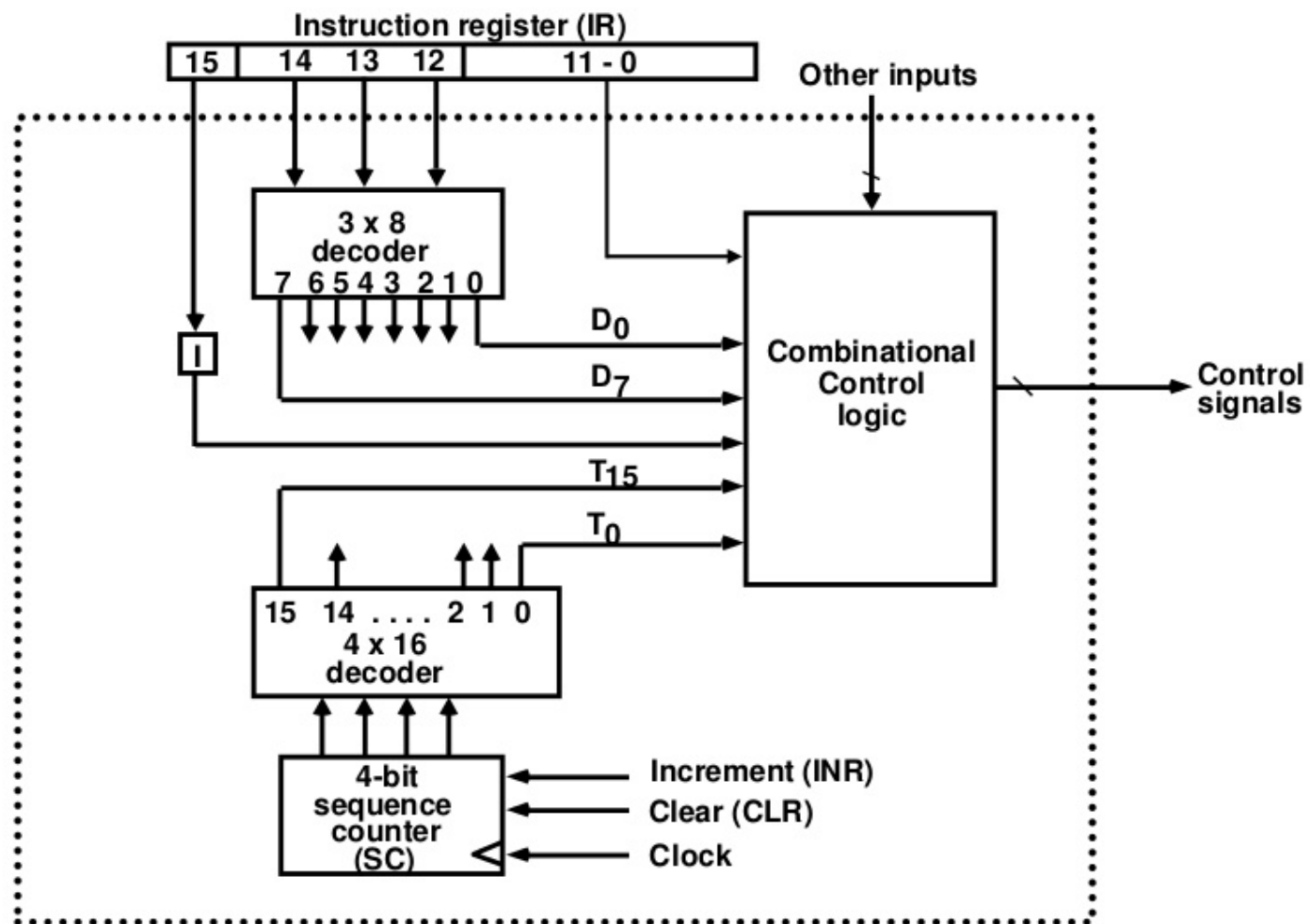
- Input and output
 - INP, OUT

CONTROL UNIT

- Control unit (CU) of a processor translates from machine instructions to the control signals for the microoperations that implement them
- Control units are implemented in one of two ways
- **Hardwired Control**
 - CU is made up of sequential and combinational circuits to generate the control signals
- **Microprogrammed Control**
 - A control memory on the processor contains microprograms that activate the necessary control signals
- We will consider a hardwired implementation of the control unit for the Basic Computer

TIMING AND CONTROL

Control unit of Basic Computer



INSTRUCTION CYCLE

- **In Basic Computer, a machine instruction is executed in the following cycle:**
 1. Fetch an instruction from memory
 2. Decode the instruction
 3. Read the effective address from memory if the instruction has an indirect address
 4. Execute the instruction
- **After an instruction is executed, the cycle starts again at step 1, for the next instruction**
- ***Note:* Every different processor has its own (different) instruction cycle**

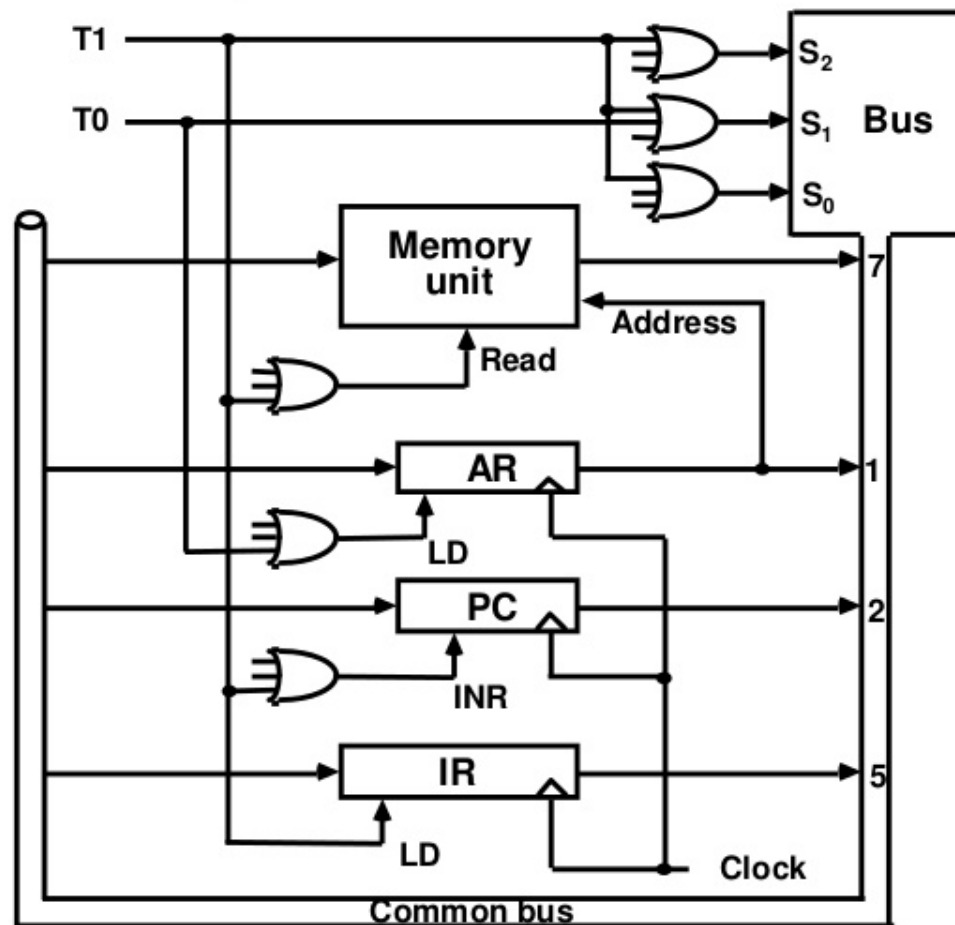
FETCH and DECODE

- Fetch and Decode

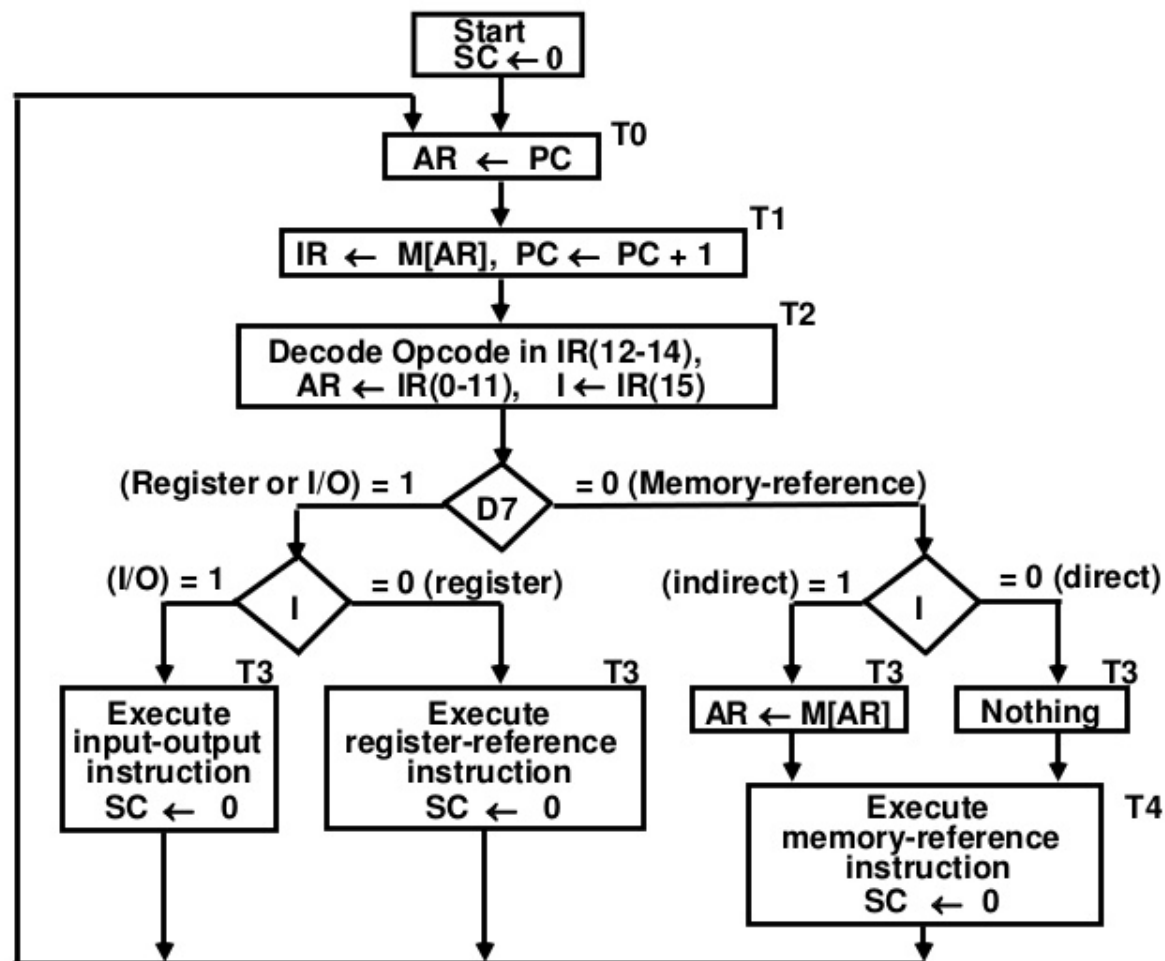
T0: $AR \leftarrow PC$ ($S_0S_1S_2=010$, $T_0=1$)

T1: $IR \leftarrow M[AR]$, $PC \leftarrow PC + 1$ ($S_0S_1S_2=111$, $T_1=1$)

T2: $D_0, \dots, D_7 \leftarrow \text{Decode } IR(12-14)$, $AR \leftarrow IR(0-11)$, $I \leftarrow IR(15)$



DETERMINE THE TYPE OF INSTRUCTION



D₇I₃: AR ← M[AR]
 D₇I₃: Nothing
 D₇I₃: Execute a register-reference instr.
 D₇I₃: Execute an input-output instr.

REGISTER REFERENCE INSTRUCTIONS

Register Reference Instructions are identified when

- $D_7 = 1, I = 0$
- Register Ref. Instr. is specified in $b_0 \sim b_{11}$ of IR
- Execution starts with timing signal T_3

$r = D_7 I' T_3 \Rightarrow$ Register Reference Instruction
 $B_i = IR(i), i=0,1,2,\dots,11$

	r:	$SC \leftarrow 0$
CLA	rB_{11} :	$AC \leftarrow 0$
CLE	rB_{10} :	$E \leftarrow 0$
CMA	rB_9 :	$AC \leftarrow AC'$
CME	rB_8 :	$E \leftarrow E'$
CIR	rB_7 :	$AC \leftarrow shr AC, AC(15) \leftarrow E, E \leftarrow AC(0)$
CIL	rB_6 :	$AC \leftarrow shl AC, AC(0) \leftarrow E, E \leftarrow AC(15)$
INC	rB_5 :	$AC \leftarrow AC + 1$
SPA	rB_4 :	if $(AC(15) = 0)$ then $(PC \leftarrow PC+1)$
SNA	rB_3 :	if $(AC(15) = 1)$ then $(PC \leftarrow PC+1)$
SZA	rB_2 :	if $(AC = 0)$ then $(PC \leftarrow PC+1)$
SZE	rB_1 :	if $(E = 0)$ then $(PC \leftarrow PC+1)$
HLT	rB_0 :	$S \leftarrow 0$ (S is a start-stop flip-flop)

MEMORY REFERENCE INSTRUCTIONS

Symbol	Operation Decoder	Symbolic Description
AND	D ₀	$AC \leftarrow AC \wedge M[AR]$
ADD	D ₁	$AC \leftarrow AC + M[AR], E \leftarrow C_{out}$
LDA	D ₂	$AC \leftarrow M[AR]$
STA	D ₃	$M[AR] \leftarrow AC$
BUN	D ₄	$PC \leftarrow AR$
BSA	D ₅	$M[AR] \leftarrow PC, PC \leftarrow AR + 1$
ISZ	D ₆	$M[AR] \leftarrow M[AR] + 1, \text{ if } M[AR] + 1 = 0 \text{ then } PC \leftarrow PC + 1$

- The effective address of the instruction is in AR and was placed there during timing signal T₂ when I = 0, or during timing signal T₃ when I = 1
- Memory cycle is assumed to be short enough to complete in a CPU cycle
- The execution of MR instruction starts with T₄

AND to AC

D₀T₄: DR $\leftarrow$ M[AR] Read operand

D₀T₅: AC $\leftarrow$ AC $\wedge$ DR, SC $\leftarrow$ 0 AND with AC

ADD to AC

D₁T₄: DR $\leftarrow$ M[AR] Read operand

D₁T₅: AC $\leftarrow$ AC + DR, E $\leftarrow$ C_{out}, SC $\leftarrow$ 0 Add to AC and store carry in E

MEMORY REFERENCE INSTRUCTIONS

LDA: Load to AC

$D_2T_4: DR \leftarrow M[AR]$

$D_2T_5: AC \leftarrow DR, SC \leftarrow 0$

STA: Store AC

$D_3T_4: M[AR] \leftarrow AC, SC \leftarrow 0$

BUN: Branch Unconditionally

$D_4T_4: PC \leftarrow AR, SC \leftarrow 0$

BSA: Branch and Save Return Address

$M[AR] \leftarrow PC, PC \leftarrow AR + 1$

BSA:

$D_5T_4: M[AR] \leftarrow PC, AR \leftarrow AR + 1$

$D_5T_5: PC \leftarrow AR, SC \leftarrow 0$

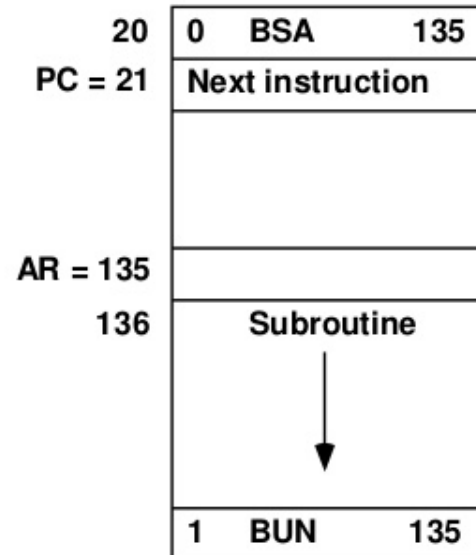
ISZ: Increment and Skip-if-Zero

$D_6T_4: DR \leftarrow M[AR]$

$D_6T_5: DR \leftarrow DR + 1$

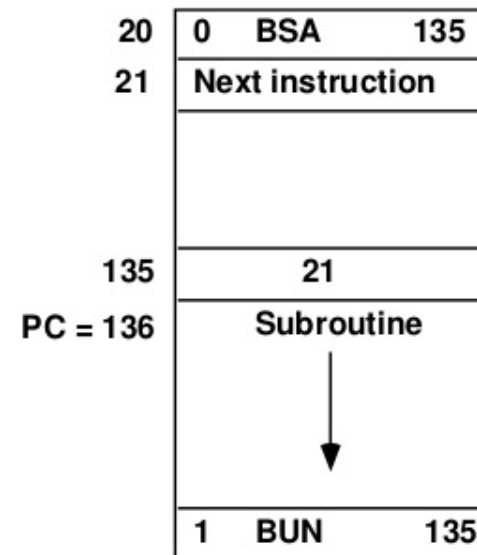
$D_6T_4: M[AR] \leftarrow DR, \text{ if } (DR = 0) \text{ then } (PC \leftarrow PC + 1), SC \leftarrow 0$

Memory, PC, AR at time T4



Memory

Memory, PC after execution

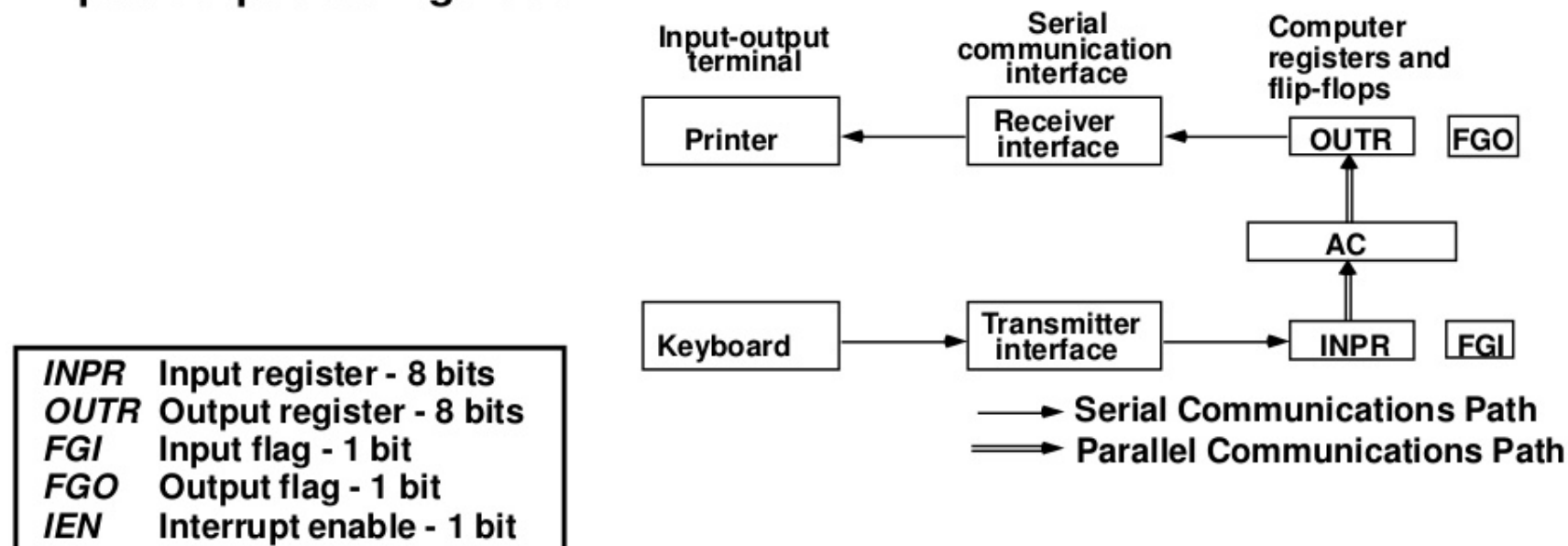


Memory

INPUT-OUTPUT AND INTERRUPT

A Terminal with a keyboard and a Printer

• Input-Output Configuration



- The terminal sends and receives serial information
- The serial info. from the keyboard is shifted into INPR
- The serial info. for the printer is stored in the OUTR
- INPR and OUTR communicate with the terminal serially and with the AC in parallel.
- The flags are needed to **synchronize** the timing difference between I/O device and the computer

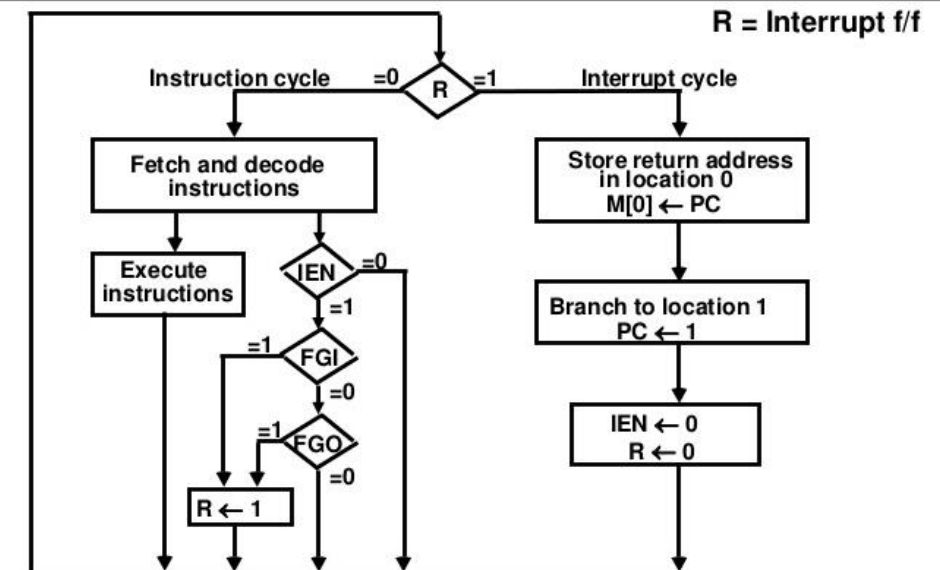
INTERRUPT INITIATED INPUT/OUTPUT

- Open communication only when some data has to be passed --> *interrupt*.
- The I/O interface, instead of the CPU, monitors the I/O device.
- When the interface finds that the I/O device is ready for data transfer, it generates an interrupt request to the CPU
- Upon detecting an interrupt, the CPU stops momentarily the task it is doing, branches to the service routine to process the data transfer, and then returns to the task it was performing.

FLOWCHART FOR INTERRUPT

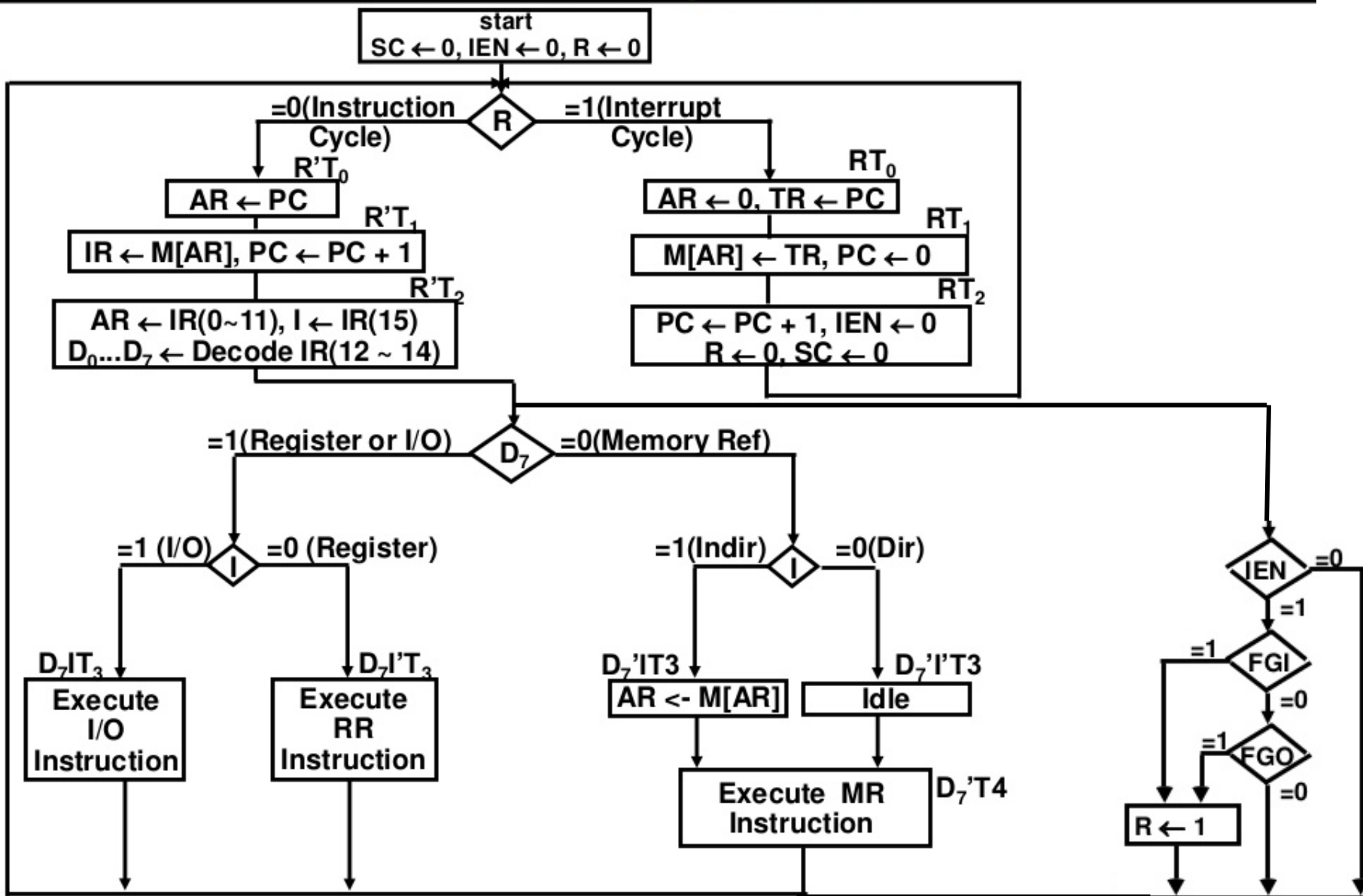
* IEN (Interrupt-enable flip-flop)

- can be set and cleared by instructions
- when cleared, the computer cannot be interrupted



COMPLETE COMPUTER DESCRIPTION

Flowchart of Operations



CONTROL OF REGISTERS AND MEMORY

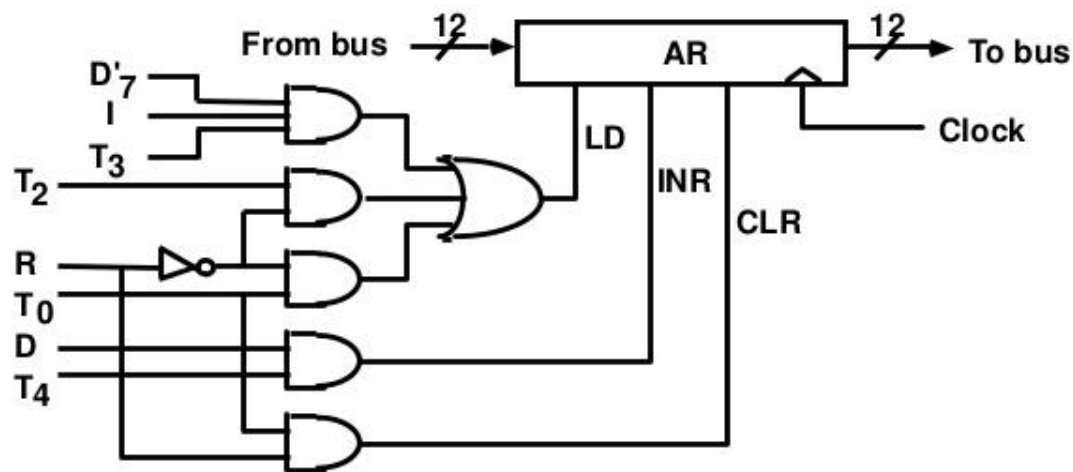
Address Register; AR

Scan all of the register transfer statements that change the content of AR:

$R'T_0$:	$AR \leftarrow PC$	$LD(AR)$
$R'T_2$:	$AR \leftarrow IR(0-11)$	$LD(AR)$
D'_7IT_3 :	$AR \leftarrow M[AR]$	$LD(AR)$
RT_0 :	$AR \leftarrow 0$	$CLR(AR)$
D_5T_4 :	$AR \leftarrow AR + 1$	$INR(AR)$

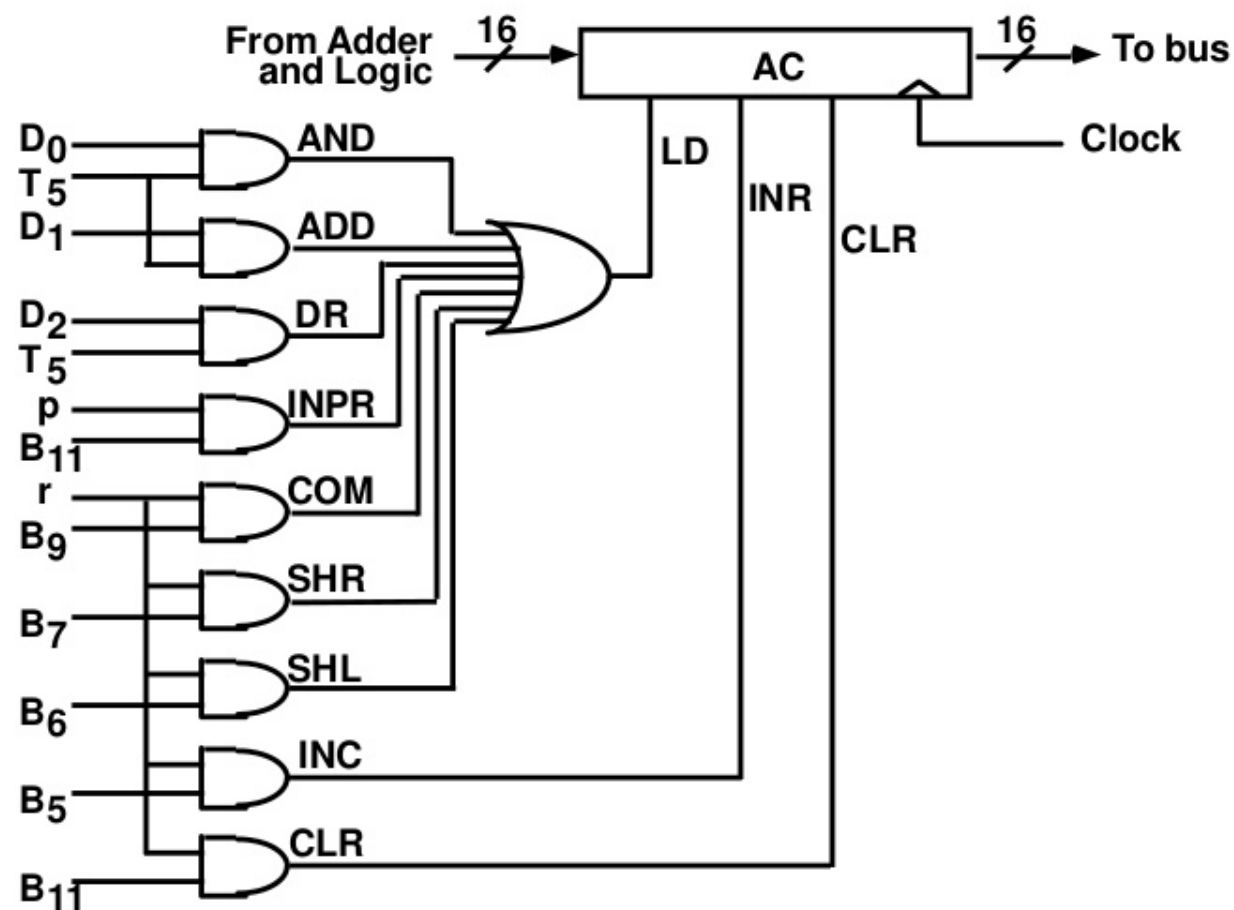


$LD(AR) = R'T_0 + R'T_2 + D'_7IT_3$
$CLR(AR) = RT_0$
$INR(AR) = D_5T_4$



CONTROL OF AC REGISTER

Gate structures for controlling the LD, INR, and CLR of AC



CONTROL OF AC REGISTER

Gate structures for controlling the LD, INR, and CLR of AC

